THE FABRIC & DATABRICKS SERIES

The SQL Handbook

for Fabric & Databricks

A practical reference for data engineers and analysts. Covers T-SQL vs Spark SQL dialect differences, Delta Lake SQL, window functions, performance tuning, platform-specific patterns, and security — for every query you write on Fabric Lakehouse, Warehouse, or Databricks SQL Warehouse.

AUTHOR

Sanjay Chandra

Data Engineering Practitioner

60+

REFERENCE PAGES

PART I

SQL Foundations

Dialects, engines, and the core mechanics that every query in Fabric and Databricks is built on.

PREREQUISITE KNOWLEDGE

Dialect Comparison; What Differs and Why It Matters

FABRIC DATABRICKS

THREE DIALECTS IN ONE ECOSYSTEM

T-SQL is Microsoft's dialect used by Fabric Warehouse. **Spark SQL** is used by Databricks SQL Warehouse and `spark.sql()` calls. **ANSI SQL** is the standard both approximate. Most SELECT / JOIN / GROUP BY / window function syntax is compatible. Differences surface in: date arithmetic, string functions, procedural extensions, NULL edge cases, and DDL.

KEY SYNTAX DIFFERENCES

Feature	T-SQL (Fabric Warehouse)	Spark SQL (Databricks)
Top N rows	SELECT TOP 10 ...	SELECT ... LIMIT 10
String concat	col1 + col2	col1 || col2
Date add	DATEADD(day, 7, col)	DATE_ADD(col, 7)
If-else inline	IIF(cond, a, b)	IF(cond, a, b)
String split	STRING_SPLIT(col, ',')	SPLIT(col, ',')
Temp tables	CREATE TABLE #tmp ...	CREATE OR REPLACE TEMP VIEW tmp AS ...
Pivot	Native PIVOT ... FOR ... IN	Manual CASE + GROUP BY
Safe cast	TRY_CAST(col AS INT)	TRY_CAST(col AS INT) (DBX 12.2+)

WHAT IS ANSI-PORTABLE (WORKS ON BOTH)

SELECT / FROM / WHERE / GROUP BY / HAVING / ORDER BY, all JOIN types, UNION / INTERSECT / EXCEPT, all window functions (OVER, PARTITION BY, frame clause), CASE WHEN, COALESCE, NULLIF, COUNT / SUM / AVG / MIN / MAX, and WITH (CTEs). Write ANSI where possible.

FABRIC SQL ENDPOINT VS WAREHOUSE

Lakehouse SQL endpoint: read-only, Spark SQL dialect, auto-generated from OneLake Delta tables. **Fabric Warehouse**: read-write, T-SQL dialect, full DML and stored procedures.

> **Rule of thumb:** SQL endpoint for exploration and BI reads. Warehouse when you need DML, stored procs, or T-SQL-specific features.

Execution Engines; Choosing the Right SQL Engine

FABRIC DATABRICKS

ENGINE LANDSCAPE

Engine	Platform	Dialect	Best for
Fabric SQL Endpoint	Fabric Lakehouse	Spark SQL (read-only)	Query Delta tables, BI connect
Fabric Warehouse	Fabric	T-SQL	DML, stored procs, ad-hoc SQL
Databricks Serverless SQL Warehouse	Databricks	Spark SQL	Ad-hoc, BI, dashboards
Databricks Pro SQL Warehouse	Databricks	Spark SQL	High concurrency, SLA dashboards
Databricks All-Purpose Cluster	Databricks	Spark SQL + PySpark	Notebook dev, ETL jobs
Databricks Jobs Cluster	Databricks	Spark SQL + PySpark	Scheduled production pipelines

> **Orchestrators vs engines:** Fabric Data Pipelines and Databricks Workflows are orchestrators, not SQL engines. They call the engines above via Script or Notebook activities. Don't confuse the orchestrator with the engine running the query.

SERVERLESS VS PROVISIONED TRADE-OFF

Serverless: no cluster management, instant start, billed per query second. Best for unpredictable workloads. **Pro Warehouse**: pre-warmed, predictable latency at high concurrency. Best for dashboards with SLA requirements. Databricks Serverless SQL starts in under 5 seconds and scales to zero automatically.

DECISION GUIDE

Scenario	Engine to use
Analyst queries Fabric Lakehouse data in SQL	Fabric SQL Endpoint
Need DML / stored procedures in Fabric	Fabric Warehouse
Databricks BI dashboard (Power BI, Tableau)	Databricks Serverless SQL Warehouse
Databricks ETL pipeline running SQL transformations	All-Purpose or Jobs Cluster
Databricks high-concurrency reporting (50+ users)	Pro SQL Warehouse

SQL vs PySpark; Choosing the Right Tool

FABRIC DATABRICKS

WHEN SQL WINS

- **Analytical queries**: aggregations, joins, window functions over structured data.
- **BI and reporting**: Power BI and Tableau connect via JDBC/ODBC; they run SQL, not PySpark.
- **Simple transformations**: filter + join + group. Less boilerplate than DataFrames.
- **Stored procedures and T-SQL logic** (Fabric Warehouse): branching, error handling.
- **Shared team access**: SQL is readable by analysts who don't know Python.

WHEN PYSPARK WINS

- **Complex data engineering**: multi-step transforms, custom logic, iterative algorithms.
- **Streaming**: Structured Streaming is a PySpark API with more flexibility than Spark SQL streaming.
- **ML pipelines**: feature engineering, model training with MLlib or scikit-learn.
- **Dynamic schema handling**: column names or types as runtime variables.

MIXING BOTH IN NOTEBOOKS

SQL

```
# PySpark to load and register a temp view
df = spark.read.format('delta').load('abfss://container@storage.dfs.core.windows.net/raw/orders')
df.filter('amount > 0').createOrReplaceTempView('orders_clean')

# SQL for the analytical transformation — cleaner than PySpark for this
result = spark.sql("""
    SELECT customer_id,
           SUM(amount) AS total_spend,
           RANK() OVER (ORDER BY SUM(amount) DESC) AS spend_rank
    FROM orders_clean GROUP BY customer_id
""")
```

> **Anti-pattern:** Rewriting a clean 5-line SQL query as 30 lines of PySpark adds maintenance cost with no performance gain. Catalyst and Photon make Spark SQL fast.

Core Query Mechanics; Execution Order and Aggregation Patterns

FABRIC DATABRICKS

EXECUTION ORDER

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT / TOP. Key consequence: you cannot reference a SELECT alias in WHERE (WHERE runs before SELECT). You can reference it in ORDER BY (both platforms allow this).

WHERE VS HAVING

WHERE filters rows before grouping. **HAVING** filters groups after aggregation.

SQL

```sql
SELECT customer_id,
       COUNT(*)    AS order_count,
       AVG(amount) AS avg_amount
FROM   orders
WHERE  status = 'completed'      -- row-level, runs before GROUP BY
GROUP  BY customer_id
HAVING COUNT(*) > 5              -- group-level, runs after GROUP BY
   AND AVG(amount) > 100
ORDER  BY avg_amount DESC;
```

ROLLUP AND GROUPING SETS

SQL

```sql
-- ROLLUP: totals by year, then year+month, then grand total
SELECT year, month, SUM(revenue) AS rev
FROM   sales
GROUP  BY ROLLUP(year, month);

-- GROUPING SETS: choose exactly which combinations
SELECT region, category, SUM(sales)
FROM   facts
GROUP  BY GROUPING SETS ((region, category), (region), ());
```

TOP VS LIMIT

T-SQL: `SELECT TOP 10 ...` placed after SELECT. Spark SQL: `... LIMIT 10` at the end. Always pair LIMIT / TOP with an ORDER BY for deterministic results.

> **T-SQL gotcha:** SELECT TOP 10 without ORDER BY returns an arbitrary 10 rows.

NULL Handling; Semantics, Aggregation, and NaN vs NULL

FABRIC DATABRICKS

NULL SEMANTICS

NULL means unknown. Any arithmetic or comparison with NULL returns NULL. `NULL = NULL` is NULL, not TRUE. Use `IS NULL` / `IS NOT NULL`. Consistent across T-SQL and Spark SQL.

COALESCE AND NULLIF

SQL

```
-- COALESCE: first non-NULL value
SELECT COALESCE(phone, email, 'no contact') AS contact
FROM   customers;

-- NULLIF: avoid division by zero
SELECT revenue / NULLIF(units, 0) AS revenue_per_unit
FROM   sales;
```

NULL IN AGGREGATIONS

Expression	If all values NULL	Notes
COUNT(*)	Returns row count	Never NULL
COUNT(col)	0	Counts non-NULLs only
SUM(col)	NULL	Wrap in COALESCE if 0 expected
AVG(col)	NULL	Ignores NULLs in denominator
MIN / MAX(col)	NULL	Returns NULL on empty input

SPARK SQL: NAN VS NULL

Spark distinguishes **NULL** (unknown) from **NaN** (not-a-number, result of 0.0/0.0). NaN is not NULL. `isnan(col)` tests for NaN. T-SQL has no NaN concept.

> **Trap:** Float columns with NaN values produce unexpected NaN aggregation results. Replace NaN at ingestion with `CASE WHEN isnan(col) THEN NULL ELSE col END`.

Data Types; T-SQL vs Spark SQL Mapping and Gotchas

FABRIC DATABRICKS

TYPE MAPPING

Concept	T-SQL (Fabric Warehouse)	Spark SQL (Databricks)
Variable string	VARCHAR(n), NVARCHAR(n)	STRING (unbounded)
Integer	INT / BIGINT	INT / BIGINT
Exact decimal	DECIMAL(p, s)	DECIMAL(p, s)
Boolean	BIT (0/1)	BOOLEAN (TRUE/FALSE)
Date only	DATE	DATE
Timestamp	DATETIME2(n)	TIMESTAMP
Array	Not supported natively	ARRAY<type>
Struct / Map	Not supported natively	STRUCT<...>, MAP<k,v>

CAST AND TRY_CAST

SQL

```
-- TRY_CAST: returns NULL on failure (safe) — both platforms
SELECT TRY_CAST(col AS INT) FROM tbl;

-- CAST: raises error on failure in Spark SQL (ANSI mode on)
SELECT CAST(col AS INT) FROM tbl;

-- Enable ANSI mode in Databricks for T-SQL-like type error behaviour
SET spark.sql.ansi.enabled = true;
```

STRING TYPE NOTES

Fabric Warehouse VARCHAR stores up to 8000 bytes; NVARCHAR up to 4000 UTF-16 chars. Spark SQL STRING is unbounded. `VARCHAR(MAX)` maps cleanly to STRING. NVARCHAR collation issues disappear in Spark (always Unicode).

> **ANSI mode tip:** Enable `spark.sql.ansi.enabled = true` in Databricks to make Spark SQL behave more like T-SQL for type errors. Off by default.

String Functions; T-SQL vs Spark SQL

FABRIC DATABRICKS

STRING FUNCTION COMPARISON

Operation	T-SQL	Spark SQL
Length (chars)	LEN(col)	LENGTH(col)
Position of str	CHARINDEX('x', col)	INSTR(col, 'x')
Pad left / right	No built-in	LPAD(col, 10, '0') / RPAD(col, 10, ' ')
Regex extract	No built-in	REGEXP_EXTRACT(col, pattern, group)
Regex replace	No built-in	REGEXP_REPLACE(col, pattern, repl)
Split to array	STRING_SPLIT(col, ',')	SPLIT(col, ',')
Concat with sep	CONCAT_WS(',', a, b, c)	CONCAT_WS(',', a, b, c)
Translate chars	No built-in	TRANSLATE(col, 'abc', 'xyz')

Identical on both: LOWER, UPPER, TRIM / LTRIM / RTRIM, SUBSTRING, REPLACE, CONCAT, LEFT, RIGHT.

LIKE VS RLIKE

Both platforms support `LIKE` with `%` (any chars) and `_` (one char). Spark SQL adds `RLIKE` for full regex. T-SQL has no built-in regex.

SQL

```
-- Spark SQL: regex match
SELECT col FROM tbl WHERE col RLIKE '^[A-Z]{2}[0-9]{4}$';

-- T-SQL: limited character class syntax only
SELECT col FROM tbl WHERE col LIKE '[A-Z][A-Z][0-9][0-9][0-9][0-9]';
```

WORKING WITH DELIMITED STRINGS

Splitting a comma-delimited column into rows is a common ETL task. The approach differs significantly.

SQL

```
-- T-SQL: STRING_SPLIT returns a table, join back for row context
SELECT o.order_id, s.value AS tag
FROM   orders o
       CROSS APPLY STRING_SPLIT(o.tags, ',') s;

-- Spark SQL: SPLIT returns an array; EXPLODE turns it into rows
SELECT order_id, EXPLODE(SPLIT(tags, ',')) AS tag FROM orders;
```

STRING AGGREGATION

Concatenate multiple rows into one string — the inverse of splitting.

SQL

```
-- T-SQL: STRING_AGG (SQL Server 2017+ and Fabric Warehouse)
SELECT customer_id,
```

Date & Time Functions; T-SQL vs Spark SQL

FABRIC DATABRICKS

DATE/TIME FUNCTION COMPARISON

Operation	T-SQL	Spark SQL
Current date	CAST(GETDATE() AS DATE)	CURRENT_DATE()
Current timestamp	GETDATE()	CURRENT_TIMESTAMP()
Add interval	DATEADD(day, 7, col)	DATE_ADD(col, 7)
Subtract dates	DATEDIFF(day, start, end)	DATEDIFF(end, start)
Extract part	YEAR(col) / MONTH(col)	YEAR(col) / MONTH(col)
Format date	FORMAT(col, 'yyyy-MM-dd')	DATE_FORMAT(col, 'yyyy-MM-dd')
Parse string	TRY_CONVERT(DATE, '2024-01-15')	TO_DATE('2024-01-15', 'yyyy-MM-dd')
Truncate to month	DATETRUNC(month, col)	DATE_TRUNC('month', col)
Last day of month	EOMONTH(col)	LAST_DAY(col)

DATEDIFF ARGUMENT ORDER: CRITICAL DIFFERENCE

SQL

```
-- T-SQL: DATEDIFF(unit, start, end)
SELECT DATEDIFF(day, '2024-01-01', '2024-03-01');  -- 60

-- Spark SQL: DATEDIFF(end, start) — arguments are REVERSED
SELECT DATEDIFF('2024-03-01', '2024-01-01');       -- 60
-- Reversing silently returns -60, not an error
```

Argument order trap: DATEDIFF in T-SQL is (unit, start, end). In Spark SQL it is (end, start). Reversing gives a negative number with no error.

TIMEZONE HANDLING

Always store timestamps in UTC; apply timezone display logic at the reporting layer. T-SQL: `AT TIME ZONE 'Eastern Standard Time'`. Spark SQL: `CONVERT_TIMEZONE('UTC', 'America/New_York', col)` (Databricks 11.3+).

PART II

Joins & Set Operations

Every join type, semi and anti joins, broadcast hints, skew handling, and set operations.

CORE SQL SKILLS

Standard Joins; Types, Syntax, and NULL in Keys

FABRIC DATABRICKS

JOIN TYPE REFERENCE

Join	Returns	No match on right?
INNER JOIN	Matching rows only	Row excluded
LEFT JOIN	All left + matching right	NULLs for right columns
RIGHT JOIN	All right + matching left	NULLs for left columns
FULL OUTER JOIN	All rows from both sides	NULLs for non-matching side
CROSS JOIN	Cartesian product	N/A: no condition

STANDARD JOIN WITH FILTER

SQL

```
SELECT o.order_id, c.name, o.amount
FROM   orders  o
       LEFT JOIN customers c ON o.customer_id = c.id
WHERE  o.status = 'completed';  -- filter applied AFTER join
```

Multi-column join: `ON a.k1 = b.k1 AND a.k2 = b.k2`. Non-equi join: `ON a.start_date <= b.date AND b.date < a.end_date`.

FULL OUTER: FIND ROWS IN EITHER TABLE BUT NOT BOTH

SQL

```
SELECT COALESCE(o.order_id, i.order_id) AS order_id,
       o.amount AS order_amount,
       i.amount AS invoice_amount
FROM   orders o FULL OUTER JOIN invoices i ON o.order_id = i.order_id
WHERE  o.order_id IS NULL OR i.order_id IS NULL;
```

NULL IN JOIN KEYS

NULL never equals NULL in a join condition. Two rows with NULL in the join key will never match. Consistent in T-SQL and Spark SQL.

> **Gotcha:** A LEFT JOIN where the right key is sometimes NULL does not produce a match for those rows. Handle NULLs in keys explicitly before joining.

CROSS JOIN and Lateral Patterns

FABRIC DATABRICKS

CROSS JOIN

Produces every combination of rows. No join condition. Output rows = left count × right count. Always verify expected cardinality first.

SQL

```
-- Every product paired with every region
SELECT p.product_name, r.region_name
FROM   products p CROSS JOIN regions r;
```

CROSS APPLY / OUTER APPLY (T-SQL)

CROSS APPLY applies a table-valued expression to each left row, returning multiple rows per input. OUTER APPLY includes left rows with no output (like LEFT JOIN).

SQL

```
-- Top 2 orders per customer
SELECT c.name, o.order_id, o.amount
FROM   customers c
CROSS  APPLY (
    SELECT TOP 2 order_id, amount FROM orders
    WHERE customer_id = c.id ORDER BY amount DESC
) o;
```

LATERAL VIEW EXPLODE (SPARK SQL)

Expands an array column into rows. Common with JSON or nested data.

SQL

```
-- Explode an array column into rows
SELECT order_id, item
FROM   orders LATERAL VIEW EXPLODE(items_array) t AS item;

-- Shorthand (Spark SQL 2.2+)
SELECT order_id, EXPLODE(items_array) AS item FROM orders;
```

> **T-SQL equivalent:** Use OPENJSON or STRING_SPLIT to unnest. Fabric Warehouse supports OPENJSON for JSON arrays.

Semi and Anti Joins; Pattern, Gotcha, and Spark Syntax

FABRIC DATABRICKS

SEMI JOIN: EXISTS AND IN

Returns rows from the left table where a match exists on the right, without adding right-table columns.

SQL

```
-- EXISTS: customers who placed at least one order
SELECT c.id, c.name FROM customers c
WHERE  EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

ANTI JOIN: NOT EXISTS

Returns rows from the left where NO match exists on the right. Use NOT EXISTS over NOT IN when the right side can contain NULLs.

SQL

```
-- NOT EXISTS: customers who never ordered
SELECT c.id, c.name FROM customers c
WHERE  NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- LEFT JOIN anti-join pattern
SELECT c.id, c.name FROM customers c
       LEFT JOIN orders o ON c.id = o.customer_id
WHERE  o.customer_id IS NULL;
```

NOT IN WITH NULLS: THE SILENT TRAP

`NOT IN (subquery)` returns zero rows if the subquery contains any NULL. Because `x NOT IN (1, 2, NULL)` evaluates to UNKNOWN for every value of x.

> **Critical:** If the NOT IN subquery can return NULL, every row is silently excluded. Always use NOT EXISTS, or filter NULLs: `WHERE key NOT IN (SELECT key FROM tbl WHERE key IS NOT NULL)`

SPARK SQL ANTI JOIN SYNTAX

Spark SQL supports an explicit LEFT ANTI JOIN that is cleaner than NOT EXISTS in complex queries.

SQL

```
SELECT c.id, c.name FROM customers c
LEFT ANTI JOIN orders o ON c.id = o.customer_id;
```

Broadcast Joins; Strategies, Hints, Skew, and Salting

DATABRICKS

JOIN STRATEGIES IN SPARK SQL

Strategy	When Spark picks it	Good for
Broadcast Hash Join	One side < autoBroadcastJoinThreshold	Small dimension + large fact
Shuffle Hash Join	One side fits in memory per partition	Medium tables, non-sortable keys
Sort Merge Join	Both sides large; default for big joins	Two large tables
Broadcast Nested Loop	Non-equi join with small table	Range or inequality conditions

AUTO-BROADCAST AND MANUAL HINT

Spark auto-broadcasts tables smaller than `spark.sql.autoBroadcastJoinThreshold` (default 10 MB). Force a broadcast with a SQL hint when Spark cannot verify the size — e.g. after complex transformations.

SQL

```
SELECT /*+ BROADCAST(d) */ f.sale_id, d.month_name, f.amount
FROM   fact_sales f JOIN dim_date d ON f.date_id = d.date_id;

-- Raise auto-broadcast threshold to 100 MB
SET spark.sql.autoBroadcastJoinThreshold = 104857600;
```

DATA SKEW: SYMPTOMS AND FIXES

Skew: one join key value has far more rows than others. Symptom: one task runs 10× longer than the rest in the Spark UI Stage view.

- **AQE skew join**: `spark.sql.adaptive.skewJoin.enabled = true` (default on in DBX 7.3+). AQE detects and splits skewed partitions automatically.
- **Filter before join**: if NULL keys are the skew source, filter them out before joining.
- **Salting**: for truly skewed keys, append a random integer suffix to distribute one hot key across N partitions.

SALTING EXAMPLE

SQL

```
-- Salting: distribute skewed key across 10 buckets
-- Step 1: add salt to the large table
SELECT *, CONCAT(skewed_key, '_', FLOOR(RAND() * 10)) AS salted_key
FROM   large_table

-- Step 2: explode the small table to match all salt values
SELECT *, CONCAT(key, '_', n) AS salted_key
FROM   small_table CROSS JOIN (SELECT EXPLODE(SEQUENCE(0, 9)) AS n)
```

> **AQE first:** Try enabling AQE skew join before salting. Salting requires schema changes on both sides and is harder to maintain.

Set Operations; Combining and Comparing Result Sets

FABRIC DATABRICKS

UNION VS UNION ALL

UNION ALL: stacks rows keeping duplicates. Faster. **UNION**: stacks rows and removes duplicates (runs DISTINCT). Slower. Use UNION ALL unless deduplication is specifically needed.

SQL

```
-- UNION ALL: stack two tables (no dedup)
SELECT order_id, amount, 'online'   AS channel FROM online_orders
UNION ALL
SELECT order_id, amount, 'in_store' AS channel FROM store_orders;
```

INTERSECT AND EXCEPT

SQL

```
-- INTERSECT: emails in both lists
SELECT email FROM list_a INTERSECT SELECT email FROM list_b;

-- EXCEPT: customers who bought in 2023 but not 2024
SELECT customer_id FROM orders WHERE YEAR(order_date) = 2023
EXCEPT
SELECT customer_id FROM orders WHERE YEAR(order_date) = 2024;
```

RULES FOR SET OPERATIONS

- All SELECT lists must have the same number of columns with compatible types.
- Column names in the result come from the first SELECT.
- ORDER BY applies to the entire combined result and goes at the very end.

EXCEPT ALL AND INTERSECT ALL (SPARK SQL)

Spark SQL supports `EXCEPT ALL` and `INTERSECT ALL` which preserve duplicates (like UNION ALL). T-SQL does not support these variants.

> **When to prefer set ops vs joins:** Set operations are cleaner than self-joins when comparing the same table across time periods. EXCEPT is clearer than a LEFT JOIN ... WHERE right IS NULL for simple column comparisons.

PART III

Window Functions

Ranking, navigation, and aggregate window functions. Frame clause mechanics and top-N patterns.

INTERMEDIATE SQL SKILLS

Ranking Functions; Ties, Gaps, and Buckets

FABRIC DATABRICKS

WINDOW FUNCTION SYNTAX

`FUNCTION() OVER (PARTITION BY col ORDER BY col ROWS/RANGE BETWEEN ... AND ...)`. PARTITION BY resets the function per group. ORDER BY sets sort order within each partition. Frame clause limits which rows are included in the calculation.

ROW_NUMBER VS RANK VS DENSE_RANK

Function	Ties?	Gap after ties?	100, 100, 90 becomes
ROW_NUMBER()	No (arbitrary)	N/A	1, 2, 3
RANK()	Same rank	Yes (skips)	1, 1, 3
DENSE_RANK()	Same rank	No	1, 1, 2

SQL

```
SELECT dept, name, salary,
       ROW_NUMBER()  OVER (PARTITION BY dept ORDER BY salary DESC) AS rn,
       RANK()        OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk,
       DENSE_RANK()  OVER (PARTITION BY dept ORDER BY salary DESC) AS dense_rnk
FROM   employees;
```

NTILE: DIVIDE ROWS INTO N BUCKETS

`NTILE(4)` assigns each row to one of 4 equal-sized buckets (quartiles). Extra rows go to lower-numbered buckets.

SQL

```
SELECT customer_id, total_spend,
       NTILE(4) OVER (ORDER BY total_spend DESC) AS quartile
FROM   customer_summary;
```

PERCENT_RANK AND CUME_DIST

PERCENT_RANK: relative rank 0 to 1. Formula: (rank - 1) / (rows - 1). **CUME_DIST**: fraction of rows ≤ current row.

Top-N Per Group; ROW_NUMBER vs DENSE_RANK

FABRIC DATABRICKS

TOP-N PER GROUP PATTERN

Wrap the window function in a subquery or CTE, then filter on rank in the outer query.

SQL

```
-- Top 2 earners per department
SELECT dept, name, salary
FROM (
    SELECT dept, name, salary,
           DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
    FROM   employees
) ranked
WHERE  rnk <= 2
ORDER  BY dept, rnk;
```

ROW_NUMBER VS DENSE_RANK FOR TOP-N

Function	Ties at boundary	Effect for top-2 with tied rank-2
ROW_NUMBER	Exactly 2 rows (arbitrary tie-break)	One tied person excluded
RANK	Both rank-2 included; gap in ranks	Both included
DENSE_RANK	Both rank-2 included; no gap	Both included

TOP-1 PER GROUP: THE DEDUP PATTERN

ROW_NUMBER = 1 gives exactly one row per group.

SQL

```
-- Latest order per customer
SELECT customer_id, order_id, order_date, amount
FROM (
    SELECT *, ROW_NUMBER() OVER
        (PARTITION BY customer_id ORDER BY order_date DESC) AS rn
    FROM orders
) t WHERE rn = 1;
```

Navigation Functions; Period Comparisons and Frame Traps

FABRIC DATABRICKS

LAG AND LEAD

LAG(col, offset, default): value from offset rows before. **LEAD(col, offset, default)**: value from offset rows after. Default is returned when offset falls outside the partition.

MONTH-OVER-MONTH CHANGE

SQL

```
SELECT month, revenue,
       LAG(revenue, 1)  OVER (ORDER BY month) AS prev_rev,
       revenue - LAG(revenue, 1) OVER (ORDER BY month) AS mom_change,
       ROUND(100.0 * (revenue - LAG(revenue, 1) OVER (ORDER BY month))
             / NULLIF(LAG(revenue, 1) OVER (ORDER BY month), 0), 2) AS mom_pct
FROM   monthly_revenue;
```

FIRST_VALUE AND LAST_VALUE

FIRST_VALUE(col): first value in the frame. **LAST_VALUE(col)**: last value in the frame.

> **LAST_VALUE trap:** Default frame is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. LAST_VALUE returns the current row's value, not the partition's last. Always specify ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

SQL

```
SELECT customer_id, order_date, amount,
       LAST_VALUE(amount) OVER (
           PARTITION BY customer_id ORDER BY order_date
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS last_order_amount
FROM   orders;
```

NTH_VALUE

`NTH_VALUE(col, n)` returns the value of the nth row in the window. Available in both T-SQL and Spark SQL.

Aggregate Windows; Totals, Averages, and Frame Mechanics

FABRIC DATABRICKS

RUNNING TOTAL AND MOVING AVERAGE

SQL

```
SELECT order_date, daily_rev,
       SUM(daily_rev) OVER (
           ORDER BY order_date
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total,
       AVG(daily_rev) OVER (
           ORDER BY order_date
           ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS mov_avg_7d
FROM   daily_sales;
```

ROWS VS RANGE FRAME CLAUSE

Frame type	Definition	Effect with ties in ORDER BY
ROWS BETWEEN ...	Physical rows by position	Ties treated as separate rows
RANGE BETWEEN ...	Logical range by value	All tied values included in frame

Default when ORDER BY is present with no frame: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. For running totals and moving averages, prefer `ROWS` for deterministic behaviour.

CUMULATIVE PERCENTAGE

SQL

```
SELECT dept, name, salary,
       ROUND(100.0 *
           SUM(salary) OVER (PARTITION BY dept ORDER BY salary
                             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
           / SUM(salary) OVER (PARTITION BY dept),
       1) AS cum_pct
FROM   employees ORDER BY dept, salary;
```

PART IV

CTEs, Subqueries & Pivoting

Composable query patterns: WITH clause, recursive CTEs, correlated subqueries, and PIVOT/UNPIVOT.

INTERMEDIATE SQL SKILLS

CTEs; Readable Multi-Step Query Composition

FABRIC DATABRICKS

WITH CLAUSE: NAMED SUBQUERIES

A CTE gives a name to a subquery for the duration of one statement. Multiple CTEs chain with commas.

SQL

```
WITH
customer_totals AS (
    SELECT customer_id, COUNT(*) AS orders, SUM(amount) AS spend
    FROM   orders WHERE status = 'completed'
    GROUP  BY customer_id
),
tiered AS (
    SELECT customer_id, orders, spend,
           CASE WHEN spend >= 1000 THEN 'Gold'
                WHEN spend >= 500  THEN 'Silver' ELSE 'Bronze' END AS tier
    FROM   customer_totals
)
SELECT c.name, t.tier, t.spend
FROM   tiered t JOIN customers c ON t.customer_id = c.id
ORDER  BY t.spend DESC;
```

CTES VS SUBQUERIES VS TEMP TABLES

Approach	Scope	Materialised?	Best for
CTE (WITH)	One statement	Usually not (inlined)	Readable multi-step logic
Subquery	Inline	No	Simple one-off filters
Temp table / TEMP VIEW	Session	Yes	Large intermediate results reused across queries

CTE MATERIALISATION IN SPARK SQL

Spark SQL does not materialise CTEs by default — it inlines them. A CTE referenced multiple times may be re-executed each time.

> **Performance:** For expensive CTEs referenced more than once, persist the result as a temp view or Delta table to force materialisation.

Recursive CTEs; Hierarchies, Date Series, and Cycle Safety

FABRIC DATABRICKS

RECURSIVE CTE STRUCTURE

An anchor member (base case) UNION ALL a recursive member (references the CTE itself). Engine iterates until no new rows are produced.

SQL

```
-- T-SQL: no RECURSIVE keyword needed
-- Spark SQL: WITH RECURSIVE required (DBX 9.1+)
WITH org_tree AS (
    SELECT id, name, manager_id, 0 AS depth
    FROM   employees WHERE manager_id IS NULL   -- anchor
    UNION ALL
    SELECT e.id, e.name, e.manager_id, t.depth + 1
    FROM   employees e JOIN org_tree t ON e.manager_id = t.id
)                                               -- recursive
SELECT id, name, depth FROM org_tree ORDER BY depth, name;
```

GENERATING A DATE SERIES

SQL

```
-- T-SQL: recursive date range
WITH dates AS (
    SELECT CAST('2024-01-01' AS DATE) AS dt
    UNION ALL
    SELECT DATEADD(day, 1, dt) FROM dates WHERE dt < '2024-12-31'
)
SELECT dt FROM dates OPTION (MAXRECURSION 400);

-- Spark SQL: SEQUENCE is simpler
SELECT EXPLODE(SEQUENCE(DATE '2024-01-01', DATE '2024-12-31', INTERVAL 1 DAY)) AS dt;
```

CYCLE PROTECTION

T-SQL: limit with `OPTION (MAXRECURSION n)` (default 100). Spark SQL: use a depth counter and a WHERE depth < limit as a safeguard; no built-in cycle detection.

> **Spark support:** Recursive CTEs require Databricks Runtime 9.1+.

Correlated Subqueries; When to Use and When to Rewrite

FABRIC DATABRICKS

CORRELATED SUBQUERY

References a column from the outer query. Re-evaluated per outer row. Powerful but slow on large tables.

SQL

```sql
-- Orders where amount > the customer's own average
SELECT order_id, customer_id, amount FROM orders o
WHERE  amount > (
    SELECT AVG(amount) FROM orders
    WHERE  customer_id = o.customer_id  -- outer reference
);
```

REWRITE AS WINDOW FUNCTION (PREFERRED)

Same result, one pass over the data.

SQL

```sql
SELECT order_id, customer_id, amount FROM (
    SELECT order_id, customer_id, amount,
           AVG(amount) OVER (PARTITION BY customer_id) AS cust_avg
    FROM   orders
)
WHERE  amount > cust_avg;
```

Rule: If a correlated subquery can be expressed as a window function, prefer the window function. One pass vs N subquery executions.

SCALAR SUBQUERY IN SELECT

A subquery in the SELECT list returns one value per row. Convenient but re-runs per row.

Performance: A scalar correlated subquery on 10M rows executes 10M times. Rewrite with `SUM(amount) OVER (PARTITION BY customer_id)` .

PIVOT and UNPIVOT; T-SQL Native Syntax and Dynamic PIVOT

FABRIC

T-SQL PIVOT

Rotates distinct values of a column into column headers. Native T-SQL operator.

SQL

```
SELECT *
FROM (
    SELECT region, month, revenue FROM monthly_sales
) src
PIVOT (
    SUM(revenue)
    FOR month IN ([Jan], [Feb], [Mar], [Apr], [May], [Jun])
) pvt;
```

T-SQL UNPIVOT

SQL

```
SELECT region, month, revenue FROM monthly_pivot_table
UNPIVOT (
    revenue FOR month IN (Jan, Feb, Mar, Apr, May, Jun)
) unpvt;
```

DYNAMIC PIVOT (COLUMN LIST UNKNOWN AT WRITE TIME)

SQL

```
DECLARE @cols NVARCHAR(MAX), @sql NVARCHAR(MAX);

SELECT @cols = STRING_AGG(QUOTENAME(month), ',')
FROM   (SELECT DISTINCT month FROM monthly_sales) m;

SET @sql =
    'SELECT * FROM (SELECT region,month,revenue FROM monthly_sales) src
     PIVOT (SUM(revenue) FOR month IN (' + @cols + ')) pvt';

EXEC sp_executesql @sql;
```

> **Fabric note:** PIVOT and UNPIVOT work in Fabric Warehouse (T-SQL). The Fabric SQL endpoint (Spark) does not support native PIVOT; use the CASE pattern instead.

Pivoting in Spark SQL; CASE Pattern, PySpark, and STACK

DATABRICKS

MANUAL PIVOT WITH CASE WHEN

Spark SQL has no native PIVOT keyword for SQL. Use conditional aggregation.

SQL

```
SELECT region,
       SUM(CASE WHEN month = 'Jan' THEN revenue ELSE 0 END) AS jan,
       SUM(CASE WHEN month = 'Feb' THEN revenue ELSE 0 END) AS feb,
       SUM(CASE WHEN month = 'Mar' THEN revenue ELSE 0 END) AS mar
FROM   monthly_sales GROUP BY region;
```

PYSPARK .PIVOT() FOR DYNAMIC COLUMNS

Python

```
# Explicit values list (faster; avoids extra scan)
df.groupBy('region') \
  .pivot('month', ['Jan','Feb','Mar','Apr','May','Jun']) \
  .agg(F.sum('revenue')) \
  .show()
```

Performance tip: Always pass explicit values to .pivot(). Without them, Spark does an extra scan to find distinct values.

UNPIVOT IN SPARK SQL

STACK: unpivots columns into rows. Available in Spark SQL.

SQL

```
-- STACK: unpivot jan/feb/mar into rows
SELECT region, month, revenue
FROM   monthly_pivot_table
LATERAL VIEW STACK(3,
    'Jan', jan, 'Feb', feb, 'Mar', mar
) t AS month, revenue;

-- Databricks 12.2+: ANSI UNPIVOT
SELECT region, month, revenue FROM monthly_pivot_table
UNPIVOT (revenue FOR month IN (jan AS 'Jan', feb AS 'Feb', mar AS 'Mar'));
```

PART V

Data Manipulation

INSERT, UPDATE, DELETE, MERGE, COPY INTO, and incremental load patterns including SCD.

CORE DATA ENGINEERING

DML; INSERT, UPDATE, DELETE, TRUNCATE, and CTAS

FABRIC DATABRICKS

INSERT, UPDATE, DELETE

SQL

```sql
-- INSERT ... SELECT (both platforms)
INSERT INTO target (col1, col2) SELECT col1, col2 FROM source WHERE cond;

-- UPDATE (T-SQL and Databricks Delta)
UPDATE orders SET status = 'cancelled' WHERE order_date < '2023-01-01';

-- DELETE
DELETE FROM orders WHERE status = 'test_data';
```

> **Fabric SQL endpoint:** Read-only. UPDATE, DELETE, and INSERT are not supported. Use Fabric Warehouse for DML.

TRUNCATE VS DELETE

	TRUNCATE TABLE	DELETE (no WHERE)
Speed	Fast (deallocates pages)	Slow (logs every row)
WHERE clause?	No	Yes
Resets identity?	Yes (T-SQL)	No

CREATE TABLE AS SELECT (CTAS)

SQL

```sql
-- Spark SQL (Delta)
CREATE TABLE gold.customer_summary USING DELTA AS
SELECT customer_id, SUM(amount) AS total_spend
FROM   silver.orders GROUP BY customer_id;

-- T-SQL (Fabric Warehouse)
CREATE TABLE gold.customer_summary AS
SELECT customer_id, SUM(amount) AS total_spend
FROM   silver.orders GROUP BY customer_id;
```

MERGE; Upserts, Soft Deletes, and Platform Differences

FABRIC DATABRICKS

MERGE SYNTAX

Updates matching rows, inserts new rows, optionally deletes unmatched rows — one atomic operation. Requires Delta in Databricks. Works on Warehouse tables in Fabric.

SQL

```
MERGE INTO target t USING staging s ON t.id = s.id

WHEN MATCHED AND s.updated_at > t.updated_at THEN
    UPDATE SET t.name = s.name, t.amount = s.amount,
               t.updated_at = s.updated_at

WHEN NOT MATCHED THEN
    INSERT (id, name, amount, updated_at)
    VALUES (s.id, s.name, s.amount, s.updated_at)

WHEN NOT MATCHED BY SOURCE THEN DELETE;  -- Databricks only; see bottom of page for Fabric
```

MULTIPLE WHEN MATCHED CLAUSES

First matching clause wins. Use for conditional logic such as soft deletes before updates.

SQL

```
WHEN MATCHED AND s.status = 'deleted' THEN DELETE
WHEN MATCHED THEN UPDATE SET t.col = s.col
```

MERGE PERFORMANCE IN DELTA

- Partition the target by a column in the merge condition. Spark reads only matching partitions.
- Z-ORDER the target on the merge key if not partitioned by it.
- Run `OPTIMIZE` after large merges to consolidate small files.

> **MERGE vs INSERT OVERWRITE:** For full partition refreshes, INSERT OVERWRITE is faster. Use MERGE only when you need row-level granularity.

FABRIC WAREHOUSE MERGE LIMITATION

`WHEN NOT MATCHED BY SOURCE` is supported in Databricks Delta but **not in Fabric Warehouse**. Attempting it in Fabric Warehouse will produce a syntax error. To handle source-deleted rows in Fabric, run a separate DELETE after the MERGE:

SQL

```
-- Fabric Warehouse: simulate NOT MATCHED BY SOURCE with a DELETE
MERGE INTO target t USING staging s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.col = s.col
WHEN NOT MATCHED THEN INSERT (id, col) VALUES (s.id, s.col);

-- Then delete rows absent from staging
DELETE FROM target WHERE id NOT IN (SELECT id FROM staging);
```

COPY INTO; Idempotent Ingestion, Watermark, and Schema Safety

DATABRICKS

COPY INTO: IDEMPOTENT BULK LOAD

Loads files from cloud storage into a Delta table. Idempotent: re-running loads only new files. Databricks tracks which files have been loaded.

SQL

```
COPY INTO bronze.orders
FROM   'abfss://landing@storage.dfs.core.windows.net/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS  ('mergeSchema' = 'true');
```

COPY INTO VS AUTO LOADER

	COPY INTO	Auto Loader
Model	Batch, SQL-callable	Streaming, continuous
New file detection	Directory scan each run	Event-based (efficient at scale)
Scale	Thousands of files	Millions of files
Best for	Scheduled batch Bronze ingestion	Near-real-time high-volume landing

WATERMARK INCREMENTAL LOAD PATTERN

For JDBC or sources without COPY INTO: load only rows where the updated timestamp is after the last run.

SQL

```
-- Load rows newer than the last watermark
INSERT INTO silver.orders
SELECT * FROM source.orders
WHERE  updated_at > (SELECT MAX(loaded_at) FROM audit.load_log
                     WHERE  table_name = 'orders');

INSERT INTO audit.load_log VALUES ('orders', CURRENT_TIMESTAMP());
```

> **Fabric:** Fabric Data Pipelines have a built-in incremental copy activity that handles watermark tracking without SQL.

INFERSCHEMA IN PRODUCTION

`inferSchema = 'true'` scans the entire file to infer types on every run — slow and non-deterministic when source schema changes. In production, always define the target table schema first and set `inferSchema = 'false'`:

SQL

```
-- Production: explicit schema, no inference
COPY INTO bronze.orders
FROM   'abfss://landing@storage.dfs.core.windows.net/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'false')
COPY_OPTIONS  ('mergeSchema' = 'false');  -- fail on schema drift
```

SCD; Slowly Changing Dimensions with MERGE

FABRIC DATABRICKS

SCD OVERVIEW

Type	What it does	History?
Type 1	Overwrite old value	No
Type 2	Add new row; mark old as expired	Full history
Type 3	Add column for previous value	One version back

SCD TYPE 1 WITH MERGE

Assumes `dim.customers` has columns `customer_id, email, city, created_at, updated_at`.

SQL

```
MERGE INTO dim.customers t USING stg.customers s
ON t.customer_id = s.customer_id
WHEN MATCHED AND (t.email <> s.email OR t.city <> s.city) THEN
    UPDATE SET t.email = s.email, t.city = s.city,
               t.updated_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN
    INSERT (customer_id, email, city, created_at, updated_at)
    VALUES (s.customer_id, s.email, s.city,
            CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP());
```

SCD TYPE 2 (SPARK SQL)

> **Race condition risk:** Calling CURRENT_DATE() in both Step 1 and Step 2 will produce the wrong join across a midnight boundary. Capture the date once before both steps.

SQL

```
-- Spark SQL Variables require Databricks Runtime 13.1+
DECLARE VARIABLE eff_date DATE DEFAULT CURRENT_DATE();

-- Step 1: expire changed rows
MERGE INTO dim.customers t USING stg.customers s
ON  t.customer_id = s.customer_id AND t.is_current = 1
WHEN MATCHED AND (t.email <> s.email OR t.city <> s.city) THEN
    UPDATE SET t.is_current = 0, t.valid_to = eff_date;

-- Step 2: insert new current rows
INSERT INTO dim.customers (customer_id,email,city,valid_from,valid_to,is_current)
SELECT s.customer_id,s.email,s.city,eff_date,NULL,1
FROM   stg.customers s JOIN dim.customers t
       ON s.customer_id=t.customer_id AND t.is_current=0 AND t.valid_to=eff_date;
```

SCD TYPE 2 (FABRIC WAREHOUSE T-SQL)

T-SQL uses `DECLARE @var` instead of `DECLARE VARIABLE`. Both steps must run in the same batch so the variable is in scope.

PART VI

Delta Lake SQL

Delta table architecture, time travel, OPTIMIZE, ZORDER, VACUUM, Change Data Feed, and streaming Delta.

CORE DATA ENGINEERING

Delta Architecture; Transaction Log, Checkpoints, and Fabric

FABRIC DATABRICKS

WHAT DELTA LAKE ADDS

Delta is a storage layer on top of Parquet. It adds: **ACID transactions** (via a transaction log), **schema enforcement**, **time travel**, and **unified batch + streaming**. Every Delta table = Parquet data files + a `_delta_log/` directory of JSON commit files.

TRANSACTION LOG MECHANICS

Every write appends a new JSON file to `_delta_log/`. Each entry records which files were added, which were removed, schema, and commit metadata. Every 10 commits, Delta writes a Parquet checkpoint that consolidates the log for fast reads. Reading a Delta table: Spark reads the latest checkpoint, replays subsequent log entries, and determines the current active file set.

CREATE TABLE AND CONVERT TO DELTA

SQL

```
-- Create a Delta table with partitioning
CREATE TABLE IF NOT EXISTS silver.orders (
    order_id BIGINT, customer_id BIGINT,
    amount DECIMAL(12,2), order_date DATE
) USING DELTA PARTITIONED BY (order_date)
LOCATION 'abfss://silver@storage.dfs.core.windows.net/orders';

-- Convert existing Parquet files to Delta in-place
CONVERT TO DELTA parquet.`abfss://raw@storage.dfs.core.windows.net/orders/`;
```

USEFUL TABLE PROPERTIES

Property	What it controls	Example value
delta.logRetentionDuration	How long commit history is kept	'interval 30 days'
delta.deletedFileRetentionDuration	How long old Parquet files are kept (time travel)	'interval 7 days'
delta.enableChangeDataFeed	Enable row-level change tracking	'true'
delta.autoOptimize.optimizeWrite	Auto-compact small files on write	'true'
delta.autoOptimize.autoCompact	Compact after writes asynchronously	'true'

DELTA IN FABRIC

All Fabric Lakehouse tables are Delta by default. OneLake stores the Parquet + `_delta_log/`. You do not need `USING DELTA` in a Fabric Lakehouse — Delta is the only format. Fabric Warehouse tables are not Delta; they use a proprietary columnar format.

> **Fabric SQL endpoint:** Auto-discovers Delta tables in the Lakehouse and exposes them as read-only SQL objects. Schema changes reflect automatically without any registration step.

Time Travel; Querying History, Restoring Tables, and Fabric

DATABRICKS

QUERY PAST VERSIONS

SQL

```
-- By version number
SELECT * FROM silver.orders VERSION AS OF 5;

-- By timestamp
SELECT * FROM silver.orders TIMESTAMP AS OF '2024-03-01 00:00:00';

-- See all commits
DESCRIBE HISTORY silver.orders LIMIT 10;
```

RESTORE TABLE

SQL

```
-- Roll back to version 3
RESTORE TABLE silver.orders TO VERSION AS OF 3;

-- Roll back to a timestamp
RESTORE TABLE silver.orders TO TIMESTAMP AS OF '2024-02-15 09:00:00';
```

RESTORE does not delete history: RESTORE creates a new commit reinstating the old state. Version numbers continue incrementing.

RETENTION SETTINGS

Default retention: 30 days. Time travel only works within the retention window. After VACUUM removes old files, earlier versions are unavailable.

SQL

```
ALTER TABLE silver.orders SET TBLPROPERTIES (
    'delta.logRetentionDuration'         = 'interval 90 days',
    'delta.deletedFileRetentionDuration' = 'interval 90 days'
);
```

TIME TRAVEL ON FABRIC LAKEHOUSE

Fabric Lakehouse Delta tables also support time travel via the SQL endpoint (read-only). Syntax is identical to Databricks:

SQL

```
-- Fabric SQL endpoint: time travel on a Lakehouse table
SELECT * FROM my_lakehouse.dbo.orders VERSION AS OF 3;
SELECT * FROM my_lakehouse.dbo.orders TIMESTAMP AS OF '2024-03-01';

-- RESTORE TABLE requires a Spark session or Databricks
-- The Fabric SQL endpoint is read-only and cannot execute RESTORE TABLE
```

OPTIMIZE and ZORDER; Compaction, Data Skipping, and Liquid Clustering

DATABRICKS

OPTIMIZE: COMPACT SMALL FILES

Every INSERT / MERGE / streaming write creates new Parquet files. Small files hurt query performance. OPTIMIZE compacts them (default target: 1 GB per file).

SQL

```
OPTIMIZE silver.orders;

-- Compact only one partition
OPTIMIZE silver.orders WHERE order_date = '2024-03-01';

-- Enable auto-optimize for future writes
ALTER TABLE silver.orders SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact'   = 'true'
);
```

Predictive Optimization: On Unity Catalog with Predictive Optimization enabled, Databricks runs OPTIMIZE, VACUUM, and statistics collection automatically based on workload patterns. No table properties needed.

ZORDER AND LIQUID CLUSTERING

ZORDER reorders data within files by one or more columns so related data is physically co-located. Queries filtering on the Z-ORDER column skip more files.

SQL

```
OPTIMIZE silver.orders ZORDER BY (customer_id);

-- Liquid Clustering (Databricks 13.3+): online, incremental
-- No need to re-run OPTIMIZE after writes
CREATE TABLE silver.orders (...) USING DELTA CLUSTER BY (customer_id);
```

VACUUM: REMOVE OLD FILES

Deletes Parquet files no longer referenced by the log AND older than the retention period. Reclaims storage but removes time travel access to those versions.

SQL

```
VACUUM silver.orders DRY RUN;    -- preview
VACUUM silver.orders;             -- run (default: 7-day floor)
VACUUM silver.orders RETAIN 30 HOURS;  -- custom
```

Safety floor: VACUUM enforces a minimum retention of 168 hours (7 days). Do not disable this while concurrent readers may be on older versions.

CDF and Streaming; CDC Propagation and Trigger Modes

DATABRICKS

CHANGE DATA FEED (CDF)

Records row-level changes (INSERT, UPDATE, DELETE) as a queryable changelog. Downstream pipelines read only the delta since the last checkpoint.

SQL

```
-- Enable CDF
ALTER TABLE silver.orders
SET TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true');

-- Read changes from version 5 onwards
SELECT * FROM table_changes('silver.orders', 5);
```

CDF adds: `_change_type` (insert / update_preimage / update_postimage / delete), `_commit_version`, `_commit_timestamp`.

STREAMING DELTA TABLES

SQL

```
-- PySpark: streaming write to Delta (append mode)
df.writeStream
  .format('delta')
  .outputMode('append')
  .option('checkpointLocation', '/checkpoints/silver_orders')
  .table('silver.orders')
```

TRIGGER MODES

Trigger	Behaviour	Use case
processingTime = '1 minute'	Runs every minute	Low-latency streaming
once	One micro-batch then stops	Scheduled batch (legacy)
availableNow	All available data then stops	Modern scheduled batch streaming
continuous	Sub-second latency (experimental)	Ultra-low-latency

PART VII

Performance Tuning

Partitioning, statistics, query hints, AQE, caching, and reading query execution plans.

SENIOR SQL SKILLS

Partitioning; Pruning, Verification, Guidelines, and Liquid Clustering

FABRIC DATABRICKS

PARTITION PRUNING

Partitioning organises files into subdirectories by partition column. A WHERE clause on the partition column causes the engine to read only matching directories, skipping all others. For a 3-year table partitioned by day, a single-day filter reads 1/1095 of the data.

SQL

```
CREATE TABLE silver.events (
    event_id BIGINT, event_type STRING, event_date DATE, payload STRING
) USING DELTA PARTITIONED BY (event_date);

-- Reads only the 2024-03-01 partition; all others skipped
SELECT * FROM silver.events WHERE event_date = '2024-03-01';
```

VERIFY PRUNING IS WORKING

Check EXPLAIN output for `PartitionFilters`. If missing, pruning is not happening — often because the filter column is cast or transformed.

SQL

```
EXPLAIN SELECT * FROM silver.events WHERE event_date = '2024-03-01';
-- Look for: PartitionFilters: [isnotnull(event_date), (event_date = 2024-03-01)]

-- WRONG: casting defeats partition pruning
WHERE CAST(event_date AS STRING) = '2024-03-01'

-- RIGHT: compare same type
WHERE event_date = DATE '2024-03-01'
```

PARTITION COLUMN GUIDELINES

- Partition by columns used frequently in WHERE filters — date is the classic choice.
- Avoid high-cardinality columns (order_id, user_id): too many small partitions.
- Target partition size: 200 MB to 1 GB after OPTIMIZE.
- Don't partition on more than 2-3 columns. Use Z-ORDER or Liquid Clustering for additional filter columns.
- Never partition on a float or boolean — use integer or date types only.

FABRIC AND LIQUID CLUSTERING

Fabric Warehouse does not use file-level Hive-style partitioning. Performance relies on automatic statistics and result caching. For Fabric Lakehouse Delta tables, Delta partitioning applies normally via the SQL endpoint.

> **Liquid Clustering (DBX 13.3+):** For new Databricks tables, prefer `CLUSTER BY (col)` over static partitioning. Clustering is incremental and online — no re-partition needed when access patterns change, and it handles multiple cluster columns without the explosion of small partition directories.

Statistics and Hints; Catalyst, ANALYZE, and AQE

DATABRICKS

ANALYZE TABLE

Catalyst uses table statistics to choose join strategies and estimate cardinality. Run ANALYZE TABLE after large loads or schema changes.

SQL

```
-- Collect table-level and column-level statistics
ANALYZE TABLE silver.orders COMPUTE STATISTICS
FOR COLUMNS customer_id, order_date, amount;
```

QUERY HINTS

Hint	What it does
BROADCAST(table)	Force broadcast join for the table
MERGE(table)	Force sort-merge join
SHUFFLE_HASH(table)	Force shuffle hash join
REPARTITION(n)	Repartition output to n partitions
REBALANCE	Rebalance partitions to target size (AQE-friendly)

SQL

```
SELECT /*+ BROADCAST(d) */ f.*, d.region
FROM   fact_orders f JOIN dim_customer d ON f.customer_id = d.id;
```

AQE: ADAPTIVE QUERY EXECUTION

Enabled by default in Databricks Runtime 7.3+. Re-optimises plans at runtime after seeing actual partition sizes.

- **Dynamic partition coalescing**: merges small post-shuffle partitions into larger ones. Target size: `spark.sql.adaptive.advisoryPartitionSizeInBytes` (default 64 MB).
- **Dynamic join switching**: converts sort-merge join to broadcast join at runtime if one side proves small enough. Threshold: `spark.sql.adaptive.autoBroadcastJoinThreshold` (default 30 MB).
- **Skew join handling**: a partition is considered skewed when it is > 5× the median partition size AND > 256 MB. AQE splits it automatically.

Reading Plans and Caching; EXPLAIN, CACHE TABLE, AQE

DATABRICKS

EXPLAIN: READ THE QUERY PLAN

SQL

```
-- Physical plan
EXPLAIN SELECT * FROM silver.orders WHERE customer_id = 123;

-- Logical + physical (verbose)
EXPLAIN EXTENDED SELECT * FROM silver.orders WHERE customer_id = 123;

-- T-SQL (Fabric Warehouse)
SET SHOWPLAN_ALL ON;
SELECT * FROM orders WHERE customer_id = 123;
SET SHOWPLAN_ALL OFF;
```

WHAT TO LOOK FOR

- **FileScan with PartitionFilters**: partition pruning is working.
- **BroadcastHashJoin**: one side is broadcast (good for small tables).
- **SortMergeJoin**: both sides shuffled — expected for large table joins.
- **Exchange**: a shuffle. Multiple exchanges on the same data suggest caching or repartitioning.
- **CartesianProduct**: almost always a bug if unexpected.

CACHE TABLE

SQL

```
-- Cache a dimension table in Spark memory
CACHE TABLE silver.dim_customer;

-- Cache a query result
CACHE TABLE customer_summary AS
SELECT customer_id, SUM(amount) AS total FROM silver.orders GROUP BY customer_id;

UNCACHE TABLE silver.dim_customer;
```

WHEN CACHING HELPS VS HURTS

Scenario	Cache?
Dimension table joined in 10+ downstream queries	Yes
Full fact table (100 GB)	No: spills to disk; slower than direct read
Table that changes frequently	No: stale results
Databricks SQL interactive queries	Rely on automatic result cache

PART VIII

Platform- Specific Features

Fabric-only SQL patterns and Databricks-only SQL features you won't find in the other platform.

SENIOR SQL SKILLS

Fabric SQL; Endpoint vs Warehouse, Cross-Item Queries, and T-SQL

FABRIC

LAKEHOUSE SQL ENDPOINT VS WAREHOUSE

	Fabric SQL Endpoint	Fabric Warehouse
Dialect	Spark SQL (read-only)	T-SQL (read-write)
DML	Not supported	INSERT, UPDATE, DELETE, MERGE
Stored procedures	No	Yes
Source	Delta tables in OneLake	Warehouse-managed storage
Best for	Query Lakehouse Delta in SQL	Full T-SQL, ETL, stored procs

CROSS-ITEM QUERIES

Query across Lakehouses and Warehouses in the same workspace using three-part names.

SQL

```
-- T-SQL: join a Warehouse table with a Lakehouse shortcut
SELECT w.order_id, l.customer_name
FROM   [dbo].[orders]           w
       JOIN [my_lakehouse].[dbo].[customers] l
       ON w.customer_id = l.id;
```

> **OneLake shortcuts:** Shortcuts create virtual references to external Delta tables without copying data. A shortcut appears as a table in the SQL endpoint automatically.

DIRECT LAKE MODE (POWER BI ON FABRIC)

Direct Lake is a third Power BI query mode alongside Import and DirectQuery. It reads Delta Parquet files directly from OneLake without importing or hitting a SQL endpoint — combining Import speed with DirectQuery freshness.

- **Falls back to DirectQuery when**: complex DAX measures, RLS, or certain relationship types are used.
- **SQL relevance**: OPTIMIZE, ZORDER, and column types you define in SQL directly affect Direct Lake query speed.
- **Setup**: create a semantic model in Fabric pointing at Lakehouse Delta tables — no SQL endpoint query needed at runtime.

T-SQL FEATURES IN FABRIC WAREHOUSE

SQL

```
-- Variables
DECLARE @min DECIMAL(10,2) = 100.00;
SELECT * FROM orders WHERE amount > @min;

-- TRY/CATCH
BEGIN TRY
    INSERT INTO orders (id, amount) VALUES (1, -5);
END TRY
BEGIN CATCH PRINT ERROR_MESSAGE(); END CATCH;

-- Stored procedure
CREATE PROCEDURE usp_daily_summary @dt DATE AS BEGIN
```

Databricks SQL; Photon, Variables, and Warehouse Types

DATABRICKS

PHOTON ENGINE

Databricks's native vectorised query engine written in C++. Replaces JVM-based execution for supported operations. Enabled automatically on SQL Warehouses. Typical uplift: 2-4x for complex aggregations, joins, and scans. Check EXPLAIN output for 'PhotonResultStage' nodes to confirm it is active.

SQL VARIABLES AND IDENTIFIER CLAUSE

SQL

```
-- SQL Variables (Databricks 13.1+)
DECLARE VARIABLE min_date DATE DEFAULT '2024-01-01';
SET VARIABLE min_date = '2024-03-01';
SELECT * FROM orders WHERE order_date >= min_date;

-- IDENTIFIER: dynamic table or column names
DECLARE VARIABLE tbl STRING DEFAULT 'silver.orders';
SELECT * FROM IDENTIFIER(tbl) LIMIT 10;
```

Older runtimes: Before Databricks Runtime 13.1, true SQL Variables don't exist. In notebooks, use widget parameters: `dbutils.widgets.text('k', 'default')` in Python, then `${k}` in `%sql` cells. This is cell-level substitution, not a SQL Variable.

SQL WAREHOUSE TYPES

Type	Start time	Best for
Serverless	< 5 seconds	Ad-hoc, BI, low idle cost
Pro	~1-2 min (pre-warm available)	High-concurrency SLA dashboards
Classic	2-5 minutes	Legacy workloads — avoid for new builds

Default: Use Serverless for all new Databricks SQL workloads. Pro only if you need pre-warming or concurrency > 50 users.

PART IX

Security & Governance

Row-level and column-level security, dynamic data masking, catalog permissions, and audit lineage.

SENIOR / PRODUCTION SKILLS

Access Control; Unity Catalog, Fabric Roles, and T-SQL Grants

FABRIC DATABRICKS

UNITY CATALOG PERMISSIONS (DATABRICKS)

Three-level namespace: Catalog > Schema > Table. Permissions cascade down.

SQL

```
-- Grant SELECT on a schema to a group
GRANT SELECT ON SCHEMA silver TO `data_analysts`;

-- Grant MODIFY for an ETL service principal
GRANT MODIFY ON TABLE silver.orders TO `etl_sp`;

-- Revoke and inspect
REVOKE SELECT ON TABLE silver.orders FROM `data_analysts`;
SHOW GRANTS ON TABLE silver.orders;
```

FABRIC WORKSPACE ROLES

Role	Lakehouse	Warehouse	Admin tasks
Admin	Full	Full	Manage workspace
Member	Read/write	Read/write	Create items
Contributor	Read/write	Read/write	No access management
Viewer	Read only	Read only	None

T-SQL GRANT / REVOKE / DENY IN FABRIC WAREHOUSE

SQL

```
GRANT SELECT ON dbo.orders TO [analyst_user];
GRANT SELECT ON SCHEMA::dbo TO [analyst_role];
DENY  SELECT ON dbo.salary_data TO [analyst_user];
REVOKE SELECT ON dbo.orders FROM [analyst_user];
```

DENY in T-SQL: DENY overrides any GRANT, including grants via role membership. Use sparingly to avoid hard-to-debug permission conflicts.

Row-Level Security; T-SQL Policy, View-Based, and UC Row Filters

FABRIC DATABRICKS

RLS IN FABRIC WAREHOUSE (T-SQL SECURITY POLICY)

SQL

```
-- Step 1: filter function
CREATE FUNCTION security.fn_order_filter(@region VARCHAR(50))
RETURNS TABLE WITH SCHEMABINDING AS RETURN
    SELECT 1 AS result
    WHERE  @region = USER_NAME()
        OR IS_MEMBER('global_analysts') = 1;

-- Step 2: bind to table
CREATE SECURITY POLICY order_rls
ADD FILTER PREDICATE security.fn_order_filter(region) ON dbo.orders
WITH (STATE = ON);
```

RLS IN DATABRICKS: VIEW-BASED

SQL

```
CREATE OR REPLACE VIEW silver.orders_secure AS
SELECT o.* FROM silver.orders o
       JOIN (SELECT region FROM user_region_map
             WHERE  username = CURRENT_USER()) r
       ON o.region = r.region;

GRANT SELECT ON VIEW silver.orders_secure TO `data_analysts`;
REVOKE SELECT ON TABLE silver.orders  FROM `data_analysts`;
```

UNITY CATALOG ROW FILTERS (DATABRICKS 13.0+)

SQL

```
CREATE FUNCTION security.region_filter(region STRING)
RETURN region = CURRENT_USER()
    OR IS_ACCOUNT_GROUP_MEMBER('global_analysts');

ALTER TABLE silver.orders
SET ROW FILTER security.region_filter ON (region);
```

> **Best practice:** Prefer Unity Catalog row filters over view-based RLS. They are centrally managed, visible in the catalog, and work across all query interfaces including BI tools.

CLS and Masking; T-SQL DDM and Unity Catalog Column Masks

FABRIC DATABRICKS

COLUMN-LEVEL SECURITY

Revoke SELECT on the table; grant SELECT on a view that excludes sensitive columns. Unity Catalog (DBX 13.0+) supports declarative column masks that are more maintainable than views.

SQL

```
-- Unity Catalog column masking
CREATE FUNCTION security.mask_email(email STRING)
RETURN CASE WHEN IS_ACCOUNT_GROUP_MEMBER('pii_access') THEN email
            ELSE REGEXP_REPLACE(email, '(.{2}).*(@)', '$1***$2')
       END;

ALTER TABLE silver.customers ALTER COLUMN email SET MASK security.mask_email;
```

DYNAMIC DATA MASKING IN FABRIC WAREHOUSE

SQL

```
CREATE TABLE dbo.customers (
    customer_id INT,
    email  VARCHAR(200) MASKED WITH (FUNCTION = 'email()'),
    phone  VARCHAR(20)  MASKED WITH (FUNCTION = 'partial(0,"XXX-XXX-",4)'),
    salary DECIMAL(10,2) MASKED WITH (FUNCTION = 'random(1,100)')
);

-- Grant unmasked access to PII team
GRANT UNMASK ON dbo.customers TO [pii_team];
```

T-SQL MASKING FUNCTIONS

Function	Example output
default()	String: XXXX; Number: 0; Date: 1900-01-01
email()	aXXX@XXXX.com
partial(0, 'XXX-', 4)	XXX-1234
random(1, 100)	Random number in range

> **DDM is display-only:** Masking hides values at SELECT time but does not prevent UPDATE or inference attacks. It is a presentation control, not encryption.

PART X

SQL in Pipelines & Notebooks

Mixing SQL and PySpark in notebooks, parameterized queries, and SQL in Fabric and Databricks pipelines.

PRACTICAL ENGINEERING

SQL in Notebooks; Magic Commands, spark.sql(), and Parameterization

FABRIC DATABRICKS

%SQL MAGIC AND SPARK.SQL()

In Databricks and Fabric notebooks, `%sql` runs Spark SQL directly in a cell. `spark.sql()` runs SQL from Python and returns a DataFrame.

SQL

```
# Python cell: prepare a temp view
df = spark.read.format('delta').load('abfss://container@storage.dfs.core.windows.net/orders')
df.filter('amount > 0').createOrReplaceTempView('orders_clean')
```

SQL CELL AND CAPTURING RESULTS

SQL

```
# Use spark.sql() to continue processing in Python
result = spark.sql("""
    SELECT region, SUM(amount) AS total
    FROM   silver.orders GROUP BY region
""")
result.write.format('delta').mode('overwrite').saveAsTable('gold.region_totals')

# In a notebook %sql cell: reference widget parameters
# %sql SELECT * FROM silver.orders WHERE region = '${region}'
```

PARAMETERIZED QUERIES WITH WIDGETS (DATABRICKS)

SQL

```
dbutils.widgets.text('start_date', '2024-01-01')
dbutils.widgets.text('region', 'EMEA')

start  = dbutils.widgets.get('start_date')
region = dbutils.widgets.get('region')

result = spark.sql(f"""
    SELECT * FROM silver.orders
    WHERE  order_date >= '{start}' AND region = '{region}'
""")
```

> **SQL injection:** Do not interpolate user inputs directly into SQL strings in production pipelines. Validate or use parameterised spark.sql() with explicit substitution.

FABRIC NOTEBOOK PARAMETERS

Fabric notebooks support `%sql` and `spark.sql()` identically to Databricks. For parameterization, Fabric uses **notebookutils** rather than dbutils:

Python

```
# Fabric: read parameters passed from a pipeline
import notebookutils
```

SQL in Pipelines; Fabric Scripts, Databricks Jobs, and DLT SQL

FABRIC DATABRICKS

SQL IN FABRIC DATA PIPELINES

Fabric Data Pipelines can run SQL via the Script activity targeting a Fabric Warehouse (T-SQL) or via a Notebook activity.

SQL

```
-- Script activity: run stored procedure with pipeline parameter
EXEC usp_load_daily_summary @load_date = '2024-03-01';

-- With dynamic pipeline parameter:
-- @{pipeline().parameters.load_date}
```

SQL IN DATABRICKS WORKFLOWS

Jobs can run SQL directly as a SQL task (targeting a SQL Warehouse) or run a notebook as a Notebook task.

SQL

```
-- SQL task: full refresh of a Gold table
CREATE OR REPLACE TABLE gold.region_summary AS
SELECT region,
       SUM(amount) AS total_revenue,
       COUNT(*)    AS order_count
FROM   silver.orders WHERE status = 'completed'
GROUP  BY region;
```

SQL IN DELTA LIVE TABLES (DLT)

SQL

```
-- DLT SQL: define a Silver table
CREATE OR REFRESH LIVE TABLE silver_orders
COMMENT 'Cleaned orders'
AS SELECT order_id, customer_id, amount, order_date
   FROM   LIVE.bronze_orders
   WHERE  amount > 0 AND order_date IS NOT NULL;

-- DLT streaming table from Auto Loader
CREATE OR REFRESH STREAMING LIVE TABLE bronze_orders AS
SELECT * FROM cloud_files(
    'abfss://landing@storage.dfs.core.windows.net/orders/', 'json');
```

REFERENCE

Cheat Sheets & Gotchas

T-SQL vs Spark SQL side-by-side, gotchas, decision guides, and a glossary.

QUICK REFERENCE

DDL & Utility Commands; SHOW CREATE, DESCRIBE, CASCADE, UNDROP

FABRIC DATABRICKS

SHOW CREATE TABLE: REVERSE-ENGINEER A TABLE

Returns the exact DDL that would recreate the table, including schema, partitioning, location, and table properties. Essential for auditing, documentation, and migration.

SQL

```sql
-- Databricks: show the CREATE TABLE statement
SHOW CREATE TABLE silver.orders;

-- Fabric Warehouse T-SQL equivalent (no SHOW CREATE TABLE)
-- Use INFORMATION_SCHEMA.COLUMNS to get column details
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE, CHARACTER_MAXIMUM_LENGTH
FROM   INFORMATION_SCHEMA.COLUMNS
WHERE  TABLE_SCHEMA = 'dbo' AND TABLE_NAME = 'orders'
ORDER  BY ORDINAL_POSITION;
```

DESCRIBE: INSPECT TABLE STRUCTURE

SQL

```sql
-- Databricks
DESCRIBE          silver.orders;   -- column names and types
DESCRIBE EXTENDED silver.orders;   -- + partitioning, location, properties, stats
DESCRIBE DETAIL   silver.orders;   -- + file count, size, format, Delta version

-- Fabric Warehouse T-SQL (sp_columns / sp_help not supported)
-- Use INFORMATION_SCHEMA views
SELECT * FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'orders';
SELECT * FROM INFORMATION_SCHEMA.TABLES  WHERE TABLE_NAME = 'orders';
```

SCHEMA AND CATALOG NAVIGATION

SQL

```sql
-- Databricks (Unity Catalog three-level namespace)
SHOW CATALOGS;
SHOW SCHEMAS IN my_catalog;
SHOW TABLES  IN silver;
SHOW VIEWS   IN silver;
SHOW COLUMNS IN silver.orders;
SHOW PARTITIONS silver.events;
SHOW TBLPROPERTIES silver.orders;
USE CATALOG my_catalog;
USE SCHEMA silver;

-- Both platforms: create schema and view
CREATE SCHEMA IF NOT EXISTS gold;
CREATE OR REPLACE VIEW gold.active_orders AS
SELECT * FROM silver.orders WHERE status = 'active';
```

DDL Advanced; CLONE, Metadata, Schema Evolution, REPAIR

FABRIC DATABRICKS

CLONE: COPY A DELTA TABLE (DATABRICKS)

Deep clone: full independent copy. Shallow clone: metadata only, references original files. Use deep clone for backups, regulatory snapshots, and migrations. Use shallow clone for short-lived dev / test copies.

SQL

```sql
-- Deep clone: full independent copy
CREATE TABLE gold.orders_backup
DEEP CLONE silver.orders;

-- Shallow clone: instant, no data copy; refers to source files
CREATE TABLE gold.orders_test
SHALLOW CLONE silver.orders;

-- Clone at a specific version (point-in-time snapshot)
CREATE TABLE gold.orders_snapshot
DEEP CLONE silver.orders VERSION AS OF 10;
```

Shallow clone caveat: If the source is VACUUMed, the shallow clone may lose files it references. Use deep clone for long-lived copies.

Fabric Warehouse: Fabric Warehouse has its own zero-copy clone via `CREATE TABLE target AS CLONE OF source`. Fabric Lakehouse Delta tables do not support CLONE; use `CREATE TABLE AS SELECT` instead.

INFORMATION_SCHEMA: QUERY METADATA AS SQL

Unity Catalog exposes `INFORMATION_SCHEMA` per catalog. Fabric Warehouse exposes it per database. Same view names, slightly different scoping.

SQL

```sql
-- Databricks: scoped to one Unity Catalog (replace <catalog>)
SELECT table_name, table_type
FROM   <catalog>.information_schema.tables
WHERE  table_schema = 'silver'
ORDER  BY table_name;

-- Fabric Warehouse T-SQL: scoped to current database (no prefix)
SELECT TABLE_NAME, TABLE_TYPE
FROM   INFORMATION_SCHEMA.TABLES
WHERE  TABLE_SCHEMA = 'dbo'
ORDER  BY TABLE_NAME;
```

ALTER TABLE: ADD, RENAME, DROP COLUMNS

SQL

```sql
-- Add a column (both platforms)
ALTER TABLE silver.orders ADD COLUMN discount DECIMAL(5,2);

-- Databricks Delta
```

REFERENCE T-SQL vs Spark SQL Side-by-Side Cheat Sheet

Side-by-Side Cheat Sheet; Most-Used Patterns and Delta Reference

FABRIC DATABRICKS

T-SQL VS SPARK SQL: MOST-USED PATTERNS

Pattern	T-SQL (Fabric Warehouse)	Spark SQL (Databricks)
Top N rows	SELECT TOP 10 *	SELECT * ... LIMIT 10
Current date	CAST(GETDATE() AS DATE)	CURRENT_DATE()
Date add 7 days	DATEADD(day, 7, col)	DATE_ADD(col, 7)
Date diff	DATEDIFF(day, a, b)	DATEDIFF(b, a)
Extract year	YEAR(col)	YEAR(col)
Format date	FORMAT(col, 'yyyy-MM-dd')	DATE_FORMAT(col, 'yyyy-MM-dd')
String length	LEN(col)	LENGTH(col)
Find in string	CHARINDEX('x', col)	INSTR(col, 'x')
Split string	STRING_SPLIT(col, ',')	SPLIT(col, ',')
Regex match	LIKE (limited)	RLIKE / REGEXP_LIKE
If-else inline	IIF(cond, a, b)	IF(cond, a, b)
Boolean type	BIT (0/1)	BOOLEAN (TRUE/FALSE)
Temp table	CREATE TABLE #tmp ...	CREATE OR REPLACE TEMP VIEW tmp AS ...
NULL-safe equals	IS NOT DISTINCT FROM (SQL 2022)	a <=> b
Safe cast	TRY_CAST(col AS INT)	TRY_CAST(col AS INT) (DBX 12.2+)
Array explode	STRING_SPLIT / OPENJSON	EXPLODE(array_col)
Pivot	Native PIVOT ... FOR ... IN	Manual CASE + GROUP BY

WINDOW FUNCTIONS: IDENTICAL ON BOTH PLATFORMS

All OVER clause syntax — PARTITION BY, ORDER BY, frame clause (ROWS/RANGE BETWEEN), and all ranking / navigation / aggregate window functions — is ANSI-standard and works identically in T-SQL and Spark SQL.

> **Portable core:** Any query using only SELECT / FROM / WHERE / GROUP BY / HAVING / CASE / COALESCE / window functions / UNION / CTEs / standard JOINs is portable between both platforms with only minor adjustments.

DELTA LAKE QUICK REFERENCE

Command	What it does
DESCRIBE HISTORY tbl	Show all commits, operations, timestamps
DESCRIBE DETAIL tbl	File count, size, partition info, table format
SELECT * FROM tbl VERSION AS OF n	Time travel to version n
RESTORE TABLE tbl TO VERSION AS OF n	Roll back to version n
OPTIMIZE tbl	Compact small files (target 1 GB per file)
OPTIMIZE tbl ZORDER BY (col)	Compact + co-locate data by col for skipping
VACUUM tbl	Delete files older than retention (default 7 days)

Gotchas; The Silent Bugs That Catch Everyone

FABRIC DATABRICKS

NULL BEHAVIOR

- **NOT IN with NULLs**: returns no rows if subquery contains any NULL. Use NOT EXISTS or filter NULLs from the subquery.
- **NaN vs NULL (Spark only)**: NaN is not NULL. `AVG(col)` on a NaN column returns NaN. Replace NaN at ingestion.
- **COALESCE vs ISNULL**: `ISNULL(a, b)` is T-SQL-only. Use `COALESCE(a, b)` for portability.
- **NULL in GROUP BY**: NULLs group together on both platforms, producing a NULL group row.

CASE SENSITIVITY

- **Fabric Warehouse**: case-insensitive identifiers. `OrderID` = `orderid`.
- **Databricks with Unity Catalog**: case-insensitive identifiers. String comparisons ARE case-sensitive: `WHERE col = 'EMEA'` does not match 'emea'.
- **Databricks legacy Hive metastore**: column names are lowercased on write. Mixed-case columns from upstream sources may silently lose their case.

DATE ARITHMETIC PITFALLS

- **DATEDIFF argument order**: T-SQL is (unit, start, end). Spark SQL is (end, start). Reversing silently returns a negative number.
- **Date + integer (Spark)**: `date_col + 7` does not add 7 days. Use `DATE_ADD(col, 7)`.
- **DST transitions**: TIMESTAMP arithmetic across daylight saving boundaries can produce unexpected hour differences. Store in UTC.

COUNT(*) VS COUNT(COL) VS COUNT(DISTINCT)

Expression	Counts	Includes NULLs?
COUNT(*)	All rows	Yes
COUNT(col)	Non-NULL values of col	No
COUNT(DISTINCT col)	Distinct non-NULL values	No

> **Common bug:** Using COUNT(col) expecting a full row count: if col has NULLs, the result is lower. Use COUNT(*) to count rows.

JOIN AND FILTER ORDER PITFALLS

- **WHERE on outer join**: filtering a LEFT JOIN result on a right-table column in WHERE converts it to an INNER JOIN. Put the filter in the ON clause instead.
- **HAVING vs WHERE**: filtering on an aggregate in WHERE is a syntax error. Aggregates belong in HAVING.
- **Alias in WHERE**: SELECT aliases cannot be used in WHERE (WHERE runs before SELECT). Use a subquery or CTE.

Decision Guides; Engine Choice, Load Patterns, and Query Structure

FABRIC DATABRICKS

WHICH SQL ENGINE IN FABRIC?

Need	Engine
Query Lakehouse Delta tables in SQL	Fabric SQL Endpoint
DML — INSERT / UPDATE / DELETE / MERGE	Fabric Warehouse
Stored procedures, T-SQL procedural logic	Fabric Warehouse
BI tool connecting to Lakehouse	Fabric SQL Endpoint (XMLA/JDBC)
Cross-workspace queries	Fabric Warehouse with shortcuts

MERGE VS INSERT OVERWRITE VS COPY INTO

Approach	Use when	Cost
MERGE	Row-level upsert / SCD logic; partial source update	High: rewrites files
INSERT OVERWRITE	Full partition refresh; source replaces partition	Medium: replaces partition
COPY INTO	Bronze ingestion from files; idempotent; append-only	Low: adds new files
CREATE OR REPLACE TABLE ... AS SELECT	Full table rebuild from scratch	Low: drop and replace

CTES VS SUBQUERIES VS TEMP TABLES

Approach	Use when
CTE (WITH)	2+ logical steps; want readable named stages; single-statement scope
Subquery	Simple one-off filter; no reason to name it
Temp view / #tmp	Large intermediate result queried multiple times; spans multiple statements

> **Spark temp view:** `CREATE OR REPLACE TEMP VIEW name AS SELECT ...` is the Spark equivalent of a T-SQL temp table. Scoped to the SparkSession.

Glossary; Key Terms for SQL in Fabric and Databricks

ACID	Atomicity, Consistency, Isolation, Durability. Delta Lake provides ACID on object storage.
AQE	Adaptive Query Execution. Re-optimises Spark plans at runtime using actual partition statistics.
Auto Loader	Databricks incremental file ingestion. Detects new files via events. Preferred over COPY INTO at high volume.
Broadcast join	Spark join where the smaller table is copied to every executor, eliminating a shuffle.
Catalyst	Spark's query optimiser. Parses, analyses, and optimises logical and physical query plans.
CDF	Change Data Feed. Delta feature that records row-level changes as a queryable changelog.
CTE	Common Table Expression. Named subquery in a WITH clause, scoped to one SQL statement.
CTAS	CREATE TABLE AS SELECT. Creates a new table from a query result in a single operation.
Delta Lake	Storage layer adding ACID, time travel, schema enforcement, and unified batch/streaming on Parquet.
DLT	Delta Live Tables (Lakeflow Declarative Pipelines). Databricks framework for declarative ETL with data quality expectations.
Dynamic Data Masking	T-SQL feature (Fabric Warehouse) that redacts column values at SELECT time without changing stored data.
Liquid Clustering	Databricks 13.3+ replacement for ZORDER. Online, incremental data clustering without full table rewrites.
Medallion architecture	Bronze / Silver / Gold layered design. Bronze: raw. Silver: cleaned. Gold: business-ready aggregates.
OneLake	Fabric's unified storage layer. All Fabric items store data in OneLake — one copy, multiple access paths.
Partition pruning	Query optimisation where the engine reads only partitions relevant to a WHERE filter.
Photon	Databricks native vectorised query engine in C++. Accelerates SQL on SQL Warehouses and configured clusters.
Predicate pushdown	Optimisation applying filters at the file or storage level to minimise data read.
RLS	Row-Level Security. Restricts which rows a user sees from a table, transparently at query time.
SCD	Slowly Changing Dimension. Type 1: overwrite. Type 2: new row per version with history.
Shuffle	Spark operation redistributing data across executors by key. Expensive. Triggered by GROUP BY, JOIN, DISTINCT.
SQL Warehouse	Databricks compute for SQL. Types: Serverless (fast start), Pro (high concurrency), Classic (legacy).
Time travel	Delta feature to query a table at a past version or timestamp via VERSION AS OF / TIMESTAMP AS OF.
Unity Catalog	Databricks governance layer. Catalog > Schema > Table. Manages permissions, lineage, audit, and sharing.
VACUUM	Delta command deleting Parquet files no longer in the log and older than the retention period.
ZORDER	Delta optimisation co-locating related data within files. Improves data skipping for filtered queries.